

ScrollSense – Assignment 1

A&B

Abhijeet Kumar(DA24B001)

My Major Assumptions

I have made the following assumptions so that the database design has clear rules.

1. Handle uniqueness applies to active accounts - sec 2.1

The brief says that each user has a handle that is unique across active accounts, but it does not clearly explain what happens to the handle when an account is no longer active.

For my design, I am assuming that the uniqueness rule applies to active accounts. Historical handle values can still be stored for inactive accounts, while the current handle of an active account must be unique.

I chose this because the brief specifically says “unique across active accounts.” If handles had to remain unique across all historical accounts, then even an inactive account could prevent another user from using that handle, which would change the uniqueness constraint.

2. Handle comparison ignores case and surrounding spaces (sec 2.1)

The brief says that handles can arrive in mixed case and may contain leading or trailing spaces. It also says that support often receives complaints about handles that differ only in capitalization. It does not define exactly how the database should compare two handles.

I am assuming that leading and trailing spaces are ignored and that handle comparison is case-insensitive. For example, `Abhi`, `ABHI`, and `Abhi` would be treated as the same handle. Internal characters are not removed.

I chose this because it matches the problem described by the support team. If the comparison was case-sensitive, two users could have handles that look almost identical to people but are considered different by the database.

3. Weekly inferred-interest results are kept as history (sec 2.1)

The brief says that the ML system assigns inferred interests with a confidence score and refreshes them weekly. It does not say whether the previous result should be replaced or kept.

I am assuming that each weekly ML refresh creates a new historical inference record. The record stores the inferred interest, its confidence score, and when that result became effective. The latest result represents the current inference.

I chose this because the database has to support historical reconstruction. If the old inference was overwritten every week, we would lose what the ML system believed about the user at an earlier point in time.

4. Suppression applies to the interest category (sec 2.1)

The brief says that a user can suppress an inferred interest and that the suppression must survive future ML refreshes. It does not clearly say whether suppression belongs to one particular inference result or to the interest itself.

I am assuming that suppression is attached to the user and the interest category. Once suppressed, a future ML refresh cannot make that interest visible again unless the suppression is removed.

I chose this because otherwise a new weekly ML result could bring back an interest that the user had already suppressed. If suppression belonged only to one inference row, it would not reliably survive future refreshes.

5. A creator has one monetisation tier at a time (sec 2.2)

The brief says that a creator's monetization tier changes over time and that Finance must be able to find the tier from a previous month. It does not explicitly say whether two tiers can overlap.

I am assuming that a creator has one applicable monetization tier at any given time. When the tier changes, the previous tier ends and the new tier starts.

I chose this because Finance needs one clear answer when asking which tier a creator had at a particular time. If multiple tiers could overlap, the database would need another rule to decide which tier should be used.

6. A video can have zero or one audio track (sec 2.2)

The brief says that a video has an optional audio track and that audio tracks can be shared by many videos. It does not explicitly say whether a video can contain more than one audio track.

I am assuming that a video can have zero or one audio track, while one audio track can be used by many videos.

I chose this because it keeps the relationship between videos and audio tracks as a simple one-to-many relationship. If a video could have multiple tracks, the relationship would become many-to-many and would require a junction table.

7. A video has one current moderation state at a time (2.2)

The brief lists several moderation states and says that the state can change during the life of a video. It does not explicitly say whether multiple states can be active at the same time.

I am assuming that a video has one current moderation state at any given time. Every change is recorded as a separate historical moderation decision.

I chose this because Trust and Safety needs a clear current state when checking a video. If multiple states could be active together, the database would need another rule to decide which state controls the video.

8. A block ends an existing follow but does not erase its history (2.3)

The brief says that a block must break any existing follow in both directions. It does not say whether the existing follow should be deleted completely or recorded as something that ended.

I am assuming that the follow is recorded as having ended when the block takes effect. The historical follow relationship is kept, but it is no longer active.

I chose this because the system is required to support historical analysis. Deleting the follow would make it impossible to tell that the relationship existed before the block. If the follow was simply deleted, the database would need a different way to reconstruct the previous social graph.

A9. Every impression is stored even when it produces no view (sec 2.4)

The brief says that an impression is recorded when a clip enters a user's viewport, while a view only begins after the clip has remained on screen for 300ms. It also says that most impressions never become views.

I am assuming that every impression is stored as its own event, even when the user never watches the clip for 300ms.

I chose this because an impression and a view represent different stages of the funnel. If impressions with no view were not stored, we would not be able to measure how often content was shown but ignored. The database would also lose an important part of the recommendation funnel.

10. A retraction is stored as its own negative signal (sec 2.4)

The brief says that likes are retractable and that a retraction is a negative signal in its own right, rather than simply the absence of a like. It does not fully define whether the retraction should be stored as part of the similar record or as a separate signal event.

I am assuming that the retraction is stored as its own negative signal while the original like remains part of the interaction history.

I chose this because the ML team needs to distinguish between a user who never liked something and a user who liked it and later changed their mind. If the retraction only changed the current like state, that information would be lost.

11. Each tool call belongs to one turn and has at most one parent (sec 2.5)

The brief says that a session contains an ordered sequence of turns and that tool calls can be made before an assistant response. It also says that a tool call may trigger nested sub-calls to arbitrary depth. It does not explicitly describe whether a tool call can belong to multiple turns or multiple parent calls.

I am assuming that each tool call belongs to exactly one turn. If it is a nested call, it also has at most one immediate parent tool call.

I chose this because it gives each tool call a clear place in the execution tree. If one tool call could belong to several turns or parents, the nesting relationship would become many-to-many and would require a different relationship structure.

12. A recommendation belongs to one turn and has one position in that shelf (2.5)

The brief says that the agent can return a shelf of clips and that each recommendation must be traceable to what happened next. It gives the example of a clip being recommended at position 2 of turn 3. It does not explicitly say whether the same recommendation record can belong to multiple turns or whether its position can change.

I am assuming that each recommendation record belongs to one turn and has one position in that returned shelf. If the same clip is recommended again in another turn, it is recorded as

another recommendation event.

I chose this because the recommendation needs to be linked to what happened after that particular recommendation. If the same recommendation could be shared across several turns, it would become difficult to determine which recommendation led to the user's later watch behaviour.

Requirements Being Descoped

The following parts are intentionally not being built in this submission.

1. Full LLM and conversational search system

I am not building the actual LLM, conversational search interface, semantic search system, vector database or the complete agent orchestration layer.

The brief mentions an LLM agent that can answer questions such as “Why this?” and support conversational search. Building the actual AI system would take the project well beyond the main database problem being evaluated in Module 1. The database still needs to store the information required by the agent, but the model itself is outside the current scope.

Later, the LLM service can be connected to the database through an API. The agent can then read the stored user, recommendation, watch and interaction history and use that information when generating its responses.

2. ML model training and feature pipeline

I am not building the ML training pipeline, feature engineering system, model-serving infrastructure, or the process used to train the recommendation models.

The brief requires ML-generated interests and recommendation information, but the main purpose of this assignment is to design the database that stores and preserves the resulting information. Building the complete ML pipeline would add significant implementation work without improving the core relational design being assessed here.

The database will instead keep the outputs and their relevant timestamps so that a future ML pipeline can use the historical data for training, evaluation, or analysis. A later version can connect the database to the actual training and model-serving systems.

3. Production-scale infra

I am not implementing production-scale infrastructure like database sharding or cloud-specific deployment.

ScrollSense has a large userbase and the final production system would eventually need to handle much higher volumes than a local SQLite database. But in near future, if i have to implement, Am gonna use Postres instead of sqlite.

B. Conceptual Design

B.1. ER Diagram

The complete ER diagram is provided separately at the following link:

[View the ScrollSense Conceptual ER Diagram](#)

Entity Inventory

Identifier	Kind	One row =
User	Strong	one registered ScrollSense user.
UserHandlePeriod	Temporal	one period during which one handle belonged to one user.
UserStatusPeriod	Temporal	one period during which one account had one status.
InterestCategory	Strong	one interest category available in the system.
UserDeclaredInterest	Associative	one user's declaration of one interest category at signup.
UserInferredInterest	Temporal	one inferred-interest result for one user and one category over one validity period.
InterestSuppression	Temporal	one period during which one user suppressed one inferred-interest category.
Creator	Strong	one user who has become a creator.
CreatorTierPeriod	Temporal	one continuous period during which one creator had one monetisation tier.
Video	Strong	one uploaded video.
Hashtag	Strong	one reusable hashtag.
VideoHashtag	Associative	one association between one video and one hashtag.
AudioTrack	Strong	one reusable audio track.
ModerationDecision	Event	one moderation decision made for one video.
FollowPeriod	Temporal	one period during which one user followed another user.
BlockPeriod	Temporal	one period during which one user blocked another user.
MutePeriod	Temporal	one period during which one user muted another user.
Impression	Event	one occasion on which one video entered one user's viewport.
ViewSegment	Weak	one continuous viewing segment belonging to one impression.
Signal	Event	one explicit signal generated by one user for one video.
Session	Event	one user-agent interaction session.
Turn	Weak	one ordered turn belonging to one session.
ToolCall	Weak	one tool invocation belonging to one turn, optionally nested under another call.
PromptTemplate	Strong	one logical prompt template.
PromptTemplateVersion	Event	one immutable version snapshot of one prompt template.
Model	Strong	one named ML or LLM model.
ModelPricingPeriod	Temporal	one period during which one model had one pricing configuration.
AgentRecommendation	Event	one video recommendation returned in one turn at one shelf position.
LLMJudgeScore	Event	one LLM-judge evaluation of one agent turn.
UserRating	Event	one thumbs-up or thumbs-down rating of one agent turn.

Relationship Inventory

The main relationships represented in the ER diagram are:

- `USER` → `USER_HANDLE_PERIOD`: one user may have zero or more handle periods; each handle period belongs to exactly one user.
- `USER` → `USER_STATUS_PERIOD`: one user has one or more status periods; each status period belongs to exactly one user.
- `USER` → `USER_DECLARED_INTEREST`: one user may declare zero or more interests.
- `INTEREST_CATEGORY` → `USER_DECLARED_INTEREST`: one interest category may be declared by zero or more users.
- `USER` → `USER_INFERRED_INTEREST`: one user may have zero or more inferred-interest records.
- `INTEREST_CATEGORY` → `USER_INFERRED_INTEREST`: one category may appear in zero or more inferred-interest records.
- `USER` → `INTEREST_SUPPRESSION`: one user may have zero or more interest suppressions.
- `INTEREST_CATEGORY` → `INTEREST_SUPPRESSION`: one category may be suppressed by zero or more users.
- `USER` → `CREATOR`: a user may become zero or one creator.
- `CREATOR` → `CREATOR_TIER_PERIOD`: one creator has one or more monetisation-tier periods.
- `USER` → `VIDEO`: one user owns zero or more videos; every video has exactly one owner.
- `VIDEO` → `VIDEO_HASHTAG`: one video may have zero or more hashtag associations.
- `HASHTAG` → `VIDEO_HASHTAG`: one hashtag may be used in zero or more videos.
- `AUDIO_TRACK` → `VIDEO`: an audio track may be used by zero or more videos, while a video uses zero or one audio track.
- `VIDEO` → `MODERATION_DECISION`: every video has one or more moderation decisions, with each decision belonging to exactly one video.
- `USER` → `FOLLOW_PERIOD`: a user may follow zero or more other users.
- `USER` → `BLOCK_PERIOD`: a user may block zero or more other users.
- `USER` → `MUTE_PERIOD`: a user may mute zero or more other users.
- `USER` → `IMPRESSION`: one user may generate zero or more impressions.
- `VIDEO` → `IMPRESSION`: one video may appear in zero or more impressions.
- `IMPRESSION` → `VIEW_SEGMENT`: one impression may produce zero or more viewing segments.
- `AGENT_RECOMMENDATION` → `IMPRESSION`: one recommendation may surface zero or more impressions, while an impression may be associated with zero or one recommendation.
- `USER` → `SIGNAL`: one user may generate zero or more signals.
- `VIDEO` → `SIGNAL`: one video may receive zero or more signals.

- **SIGNAL** $\rightarrow$ **SIGNAL**: a retraction may reference one prior signal, while a like may have zero or one associated retraction.
- **USER** $\rightarrow$ **SESSION**: one user may have zero or more agent sessions.
- **SESSION** $\rightarrow$ **TURN**: each session contains one or more turns; each turn belongs to exactly one session.
- **TURN** $\rightarrow$ **TOOL_CALL**: one turn may issue zero or more tool calls.
- **TOOL_CALL** $\rightarrow$ **TOOL_CALL**: a tool call may have zero or one parent tool call and may have zero or more nested child calls.
- **PROMPT_TEMPLATE** $\rightarrow$ **PROMPT_TEMPLATE_VERSION**: one prompt template has one or more immutable versions.
- **PROMPT_TEMPLATE_VERSION** $\rightarrow$ **TURN**: one prompt-template version may be used by zero or more turns.
- **MODEL** $\rightarrow$ **TURN**: one model may be used in zero or more turns.
- **MODEL** $\rightarrow$ **MODEL_PRICING_PERIOD**: one model has one or more pricing periods.
- **TURN** $\rightarrow$ **AGENT_RECOMMENDATION**: one turn may return zero or more recommendations.
- **VIDEO** $\rightarrow$ **AGENT_RECOMMENDATION**: one video may be recommended zero or more times.
- **TURN** $\rightarrow$ **LLM_JUDGE_SCORE**: a turn may have zero or one LLM-judge score.
- **TURN** $\rightarrow$ **USER_RATING**: a turn may have zero or one user rating.

B.2 Temporal Register

Each entity is assigned exactly one of the three temporal strategies required by the task: overwrite in place, validity intervals, or append-only event log.

Entity	Strategy	Justification
User	Overwrite in place	The user identity is stable; changing account state is stored separately.
UserHandlePeriod	Validity intervals	Handles change over time and historical handle values must remain available.
UserStatusPeriod	Validity intervals	Account status changes over time and must be historically reconstructable.
InterestCategory	Overwrite in place	The category is a stable reference entity.
UserDeclaredInterest	Overwrite in place	The signup declaration is treated as a stable association.
UserInferredInterest	Validity intervals	Weekly ML refreshes require historical inference results to remain available.
InterestSuppression	Validity intervals	Suppression must survive future inference refreshes and can later change.
Creator	Overwrite in place	Creator identity is stable.
CreatorTierPeriod	Validity intervals	Finance requires the monetisation tier that was valid at a historical time.
Video	Overwrite in place	The video identity is stable.

Entity	Strategy	Justification
Hashtag	Overwrite in place	The hashtag is a reusable reference entity.
VideoHashtag	Overwrite in place	The association records hashtag membership of a video.
AudioTrack	Overwrite in place	The audio track is a stable reusable object.
ModerationDecision	Append-only event log	Every moderation decision must remain available as historical evidence.
FollowPeriod	Validity intervals	A follow can begin and end while its historical state remains queryable.
BlockPeriod	Validity intervals	A block changes over time and its historical effect must be preserved.
MutePeriod	Validity intervals	A mute relationship can begin and end and must remain historically queryable.
Impression	Append-only event log	An impression is a historical telemetry occurrence and must not be overwritten.
ViewSegment	Append-only event log	Each viewing segment represents an independent historical occurrence.
Signal	Append-only event log	Signals and like retractions are historical user actions.
Session	Append-only event log	A session represents one completed user-agent interaction occurrence.
Turn	Append-only event log	The exact user message, assistant response, and turn order are historical facts.
ToolCall	Append-only event log	Arguments, results, latency, errors, and nesting must remain historically available.
PromptTemplate	Overwrite in place	The logical template is a stable parent object.
PromptTemplateVersion	Append-only event log	Each version is an immutable snapshot referenced directly by historical turns.
Model	Overwrite in place	The model identity is stable.
ModelPricingPeriod	Validity intervals	Historical cost calculations require the price valid at the time of usage.
AgentRecommendation	Append-only event log	Each recommendation is a historical event tied to a turn and shelf position.
LLMJudgeScore	Append-only event log	A judge score records a historical evaluation of an agent response.
UserRating	Append-only event log	A thumbs-up or thumbs-down is a historical user action.

B.3 Three Hardest Design Decisions

1. How to store the moderation history

One of the hardest decisions for me was deciding how to store the moderation state of a video. At first, it seemed easier to keep the current moderation state directly with the video and also keep a separate history of previous states. However, this would mean that the same information is being stored in two places, which could cause the two values to become inconsistent.

I decided to use **MODERATION_DECISION** as an append-only record instead. Every moderation decision is stored with its timestamp, including the initial “pending” state. When I need to know the moderation state at a particular time, I can look at the latest decision known at that time. This also helps with the requirement that the database should be able to reconstruct the state of a video in the past without using information that was not available at that time.

2. How to connect recommendations with what the user watched

Another difficult part was connecting the agent’s recommendations with the watching behaviour of the user. The brief gives the example of a clip being recommended at a particular position in a turn and then asks whether the user actually watched it and for how long.

I first considered connecting **AGENT_RECOMMENDATION** directly to **VIEW_SEGMENT**. I rejected this because a view segment already comes from an impression, and giving it another direct parent would make the relationship unnecessarily complicated.

Instead, I connected them as:

$$AGENT_RECOMMENDATION \rightarrow IMPRESSION \rightarrow VIEW_SEGMENT$$

This means that an impression can record which recommendation caused the video to be shown. The view segments then record what happened after that impression. This lets the database trace a recommendation to the user’s actual viewing behaviour.

3. How to represent nested tool calls

The third difficult decision was how to represent tool calls made by the agent. A normal tool call belongs to a turn, but the brief also says that one tool call can trigger another tool call and that this nesting can continue to arbitrary depth.

I considered storing a fixed number of child tool calls, but that would put an unnecessary limit on how deeply the calls could be nested. It would also make the design harder to extend later.

I therefore used a recursive relationship for **TOOL_CALL**. Each tool call belongs to a turn and can optionally have one parent tool call. A tool call can also have multiple child tool calls. This allows the same structure to represent both simple tool calls and deeply nested tool-call sequences without adding extra columns for different nesting levels.

Design clarification: session and turn

One small clarification I made was for the relationship between **SESSION** and **TURN**. I decided that a session is created only when the first agent turn is created. This means that simply opening the agent panel does not create an empty session. Therefore, every stored session contains at least one turn.